Práctica de Laboratorio 3

Autores:

Eric Guacache C.I: 31.788.759
Renny Andrade C.I: 29.727.828

Índice

1. Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw? 2
1.1. Organización de la memoria en MM I/O 2
1.2. Región de memoria donde se suelen mapear 2
1.3. Implicaciones para las instrucciones lw y sw 2

2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O? 3
2.1. Principal Diferencia 3
2.2. Ventajas y Desventajas de cada enfoque 3
2.3. Por qué MIPS32 utiliza principalmente MMIO 3

3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto? 4
3.1. Problemas por direcciones solapadas 4
3.2. Cómo se evita el conflicto 4

4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos? 4
4.1. Simplificación del diseño del procesador 4
4.2. Instrucciones adicionales para E/S por puertos 5

5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM? 5
5.1. A nivel del bus de datos y direcciones 5
5.2. Cómo sabe el hardware que debe acceder a un periférico 5

6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados? 6
6.1. Acceso por parte de programas sin privilegios 6
6.2. Mecanismos de protección implementados 6

7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos? 6
7.1. Interrupciones por Hardware (Interrupts) 6
7.2. Acceso Directo a Memoria (DMA) 6

8. Análisis y Discusión de los Resultados 7
8.1. Implicaciones del Modelo Arquitectónico 7

8.2. Estabilidad y Protección Estructural 7
8.3. Maximización del Rendimiento 7

1. Pregunta 1

Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

1.1. Organización de la memoria en MM I/O

- **Espacio de Direcciones Unificado:** A diferencia de otros esquemas, hay un único espacio de direcciones que es compartido por la RAM, la ROM y los registros de los dispositivos I/O.
- **Registros como Ubicaciones de Memoria:** A cada registro de control o de datos de un dispositivo periférico se le asigna una dirección física única dentro del espacio de memoria.
- **Sin Instrucciones Especiales:** El procesador no necesita comandos específicos para Entrada/Salida. Las operaciones se realizan utilizando las mismas instrucciones de carga (`lw`) y almacenamiento (`sw`) empleadas para acceder a la RAM.
- **Decodificador Unificado:** Un hardware decodificador de direcciones se encarga de determinar si una dirección enviada por el procesador apunta a la RAM o a un dispositivo periférico.

1.2. Región de memoria donde se suelen mapear

Generalmente, los dispositivos I/O se mapean en regiones que se encuentran **por encima de la memoria RAM principal**, en la parte alta del espacio de direcciones físicas. Esto evita que sus direcciones se solapen con las de la memoria de sistema y aprovecha los amplios rangos de direccionamiento de las arquitecturas modernas.

1.3. Implicaciones para las instrucciones lw y sw

- **Uniformidad en el Acceso:** Permite que el software interactúe con los periféricos usando exactamente el mismo método y sintaxis que emplea para la memoria común.
- **Semántica de I/O:** Una lectura mediante `lw` se traduce en extraer el estado o dato actual del registro periférico. Una escritura mediante `sw` se utiliza para modificar su comportamiento (por ejemplo, encender un actuador o enviar un bloque de datos).
- **Gestión de Caché:** Exige que los segmentos de memoria mapeados a I/O se marquen estrictamente como **no-cacheables**, para evitar que el procesador recupere un dato antiguo desde la caché de la CPU en lugar de consultar el estado real y volátil del dispositivo.

2. Pregunta 2

¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

2.1. Principal Diferencia

En **Memory-Mapped I/O (MMIO)**, los periféricos se integran en el mismo mapa de direcciones que la memoria principal y se controlan mediante operaciones ordinarias de carga y almacenamiento. En **Port-Mapped I/O (PMIO)**, los periféricos operan en un espacio de direcciones físicamente aislado e independiente, requiriendo instrucciones de procesador exclusivas (como `IN` y `OUT`).

2.2. Ventajas y Desventajas de cada enfoque

- **Memory-Mapped I/O (MMIO):**
  - *Ventajas:* Simplifica el diseño de la CPU al no requerir un conjunto extra de instrucciones. Permite acceder a los periféricos empleando la flexibilidad completa del conjunto de instrucciones de memoria. Facilita transferencias DMA eficientes.
  - *Desventajas:* Resta espacio útil al mapa de memoria física global y obliga a una administración cuidadosa para evitar que la memoria caché guarde datos de I/O por error.
- **Entrada/Salida por Puertos (PMIO):**
  - *Ventajas:* Protege el 100 % del direccionamiento de la memoria principal exclusivamente para la RAM. Garantiza una clara barrera de seguridad de acceso a nivel de arquitectura.
  - *Desventajas:* Obliga al procesador a incorporar instrucciones exclusivas que aumentan la complejidad del silicio. Menor flexibilidad programática, ya que estas instrucciones especiales suelen admitir menos formatos y registros.

2.3. Por qué MIPS32 utiliza principalmente MMIO

La arquitectura MIPS32 es inherentemente **RISC (Reduced Instruction Set Computer)**. Su objetivo rector es la simplicidad extrema del hardware para lograr ciclos de reloj rápidos. Al adoptar MMIO, MIPS elimina todo el conjunto de instrucciones dedicado a puertos de I/O, estandarizando toda interacción bajo `lw` y `sw`. Esta uniformidad facilita el desarrollo de compiladores y optimiza la tubería (*pipeline*) del procesador.

3. Pregunta 3

En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

3.1. Problemas por direcciones solapadas

- **Corrupción generalizada de datos:** Al emitir una dirección solapada, tanto el dispositivo A como el dispositivo B intentarán responder. El procesador recibirá bits mezclados o sobreescritos.
- **Comportamiento Erróneo o Inestable:** Escribir en la dirección disparará rutinas en el dispositivo incorrecto. Esto se manifiesta en fallos de hardware intermitentes, reinicios repentinos (*kernel panics*) y bloqueos irreversibles (*hangs*).
- **Vulnerabilidades:** Representa una brecha grave de seguridad, ya que el control sobre un dispositivo inofensivo podría desencadenar modificaciones en el registro de un componente del sistema de alta criticidad.

3.2. Cómo se evita el conflicto

El diseño informático combate este solapamiento mediante un esfuerzo conjunto de tres capas:

- **Hardware:** Uso de *decodificadores de direcciones* estructurados de fábrica para emitir la señal de *Chip Select* a un solo componente lógico por rango.
- **Firmware (BIOS/UEFI):** Durante el encendido del equipo, audita los dispositivos conectados al bus (como PCIe) y les asigna rangos de direcciones Base (BARs) que están matemáticamente garantizados de no cruzarse.
- **Sistema Operativo:** Mantiene un mapa unificado estricto de los recursos y evita que los controladores reclamen memoria o puertos que ya figuran ocupados por otro periférico.

4. Pregunta 4

¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

4.1. Simplificación del diseño del procesador

En el modelo MMIO el hardware no diferencia lógicamente entre interactuar con la RAM o con un periférico. Debido a esto, la CPU puede reutilizar el 100 % de la circuitería destinada a sus instrucciones convencionales de almacenamiento y lectura de memoria, omitiendo integrar decodificadores internos, microcódigo y validaciones para comandos dedicados puramente a periféricos.

4.2. Instrucciones adicionales para E/S por puertos

Si el sistema migrara a un esquema de puertos, el procesador estaría forzado a soportar comandos adicionales estructurados (por ejemplo `IN` y `OUT`), los cuales deberían abarcar:

- **Opcodes Específicos:** Códigos de operación legibles por el procesador que identifiquen inequívocamente que la orden va dirigida al bus de Entrada/Salida.
- **Direccionamiento Paralelo:** Capacidad para declarar qué número de "puerto"físico se desea intervenir.
- **Manejo de Tamaño de Envío:** Variantes del comando para transferir información a nivel de byte, media palabra o palabra completa hacia dicho puerto.

5. Pregunta 5

¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

5.1. A nivel del bus de datos y direcciones

El procesador inicia la transacción de manera genérica: coloca en el **bus de direcciones** el valor físico del registro deseado. En el bus de control, activa el pin correspondiente a la acción requerida (*Read* o *Write*). Si se trata de una escritura, expulsa el dato procesado al **bus de datos**; si es una lectura, retiene su ciclo a la espera de que el dispositivo externo inyecte la respuesta en el bus de datos.

5.2. Cómo sabe el hardware que debe acceder a un periférico

El procesador en sí mismo **lo ignora**. La magia del enrutamiento recae totalmente en un circuito externo integrado en la placa base conocido como **decodificador de direcciones**. Este decodificador posee el mapa topológico del sistema. Cuando detecta que la numeración eléctrica viajando en el bus de direcciones coincide con el rango reservado para un dispositivo externo, envía una señal de activación exclusiva (*Chip Select*) hacia el controlador de dicho periférico, al tiempo que mantiene a la memoria RAM primaria en estado inactivo.

6. Pregunta 6

¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

6.1. Acceso por parte de programas sin privilegios

Aunque la dirección es una dirección de memoria estándar, en sistemas operativos modernos **no es posible** que el software de usuario final apunte directamente al hardware de I/O. Intentar hacerlo provocará una detención de seguridad abrupta del programa.

6.2. Mecanismos de protección implementados

- **Modos de Privilegio de la CPU:** El procesador clasifica las ejecuciones en "modo kernel"(total acceso) y "modo usuario"(acceso perimetral).
- **Unidad de Gestión de Memoria (MMU):** Este subsistema mapea la memoria virtual a la física. El sistema operativo instruye a la MMU para que marque las páginas de MMIO con permisos restringidos únicamente para el anillo del Kernel. Si un software de usuario exige la ruta, la MMU arroja un fallo (*Segmentation Fault*).
- **System Calls (Llamadas al Sistema):** Constituyen el puente legal. El programa de usuario pide de favor al Sistema Operativo que manipule el hardware. La ejecución cruza momentáneamente al modo Kernel, ejecuta el controlador seguro y retorna los resultados al espacio de usuario.

7. Pregunta 7

¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

7.1. Interrupciones por Hardware (Interrupts)

El procesador dicta la orden de inicio al dispositivo y se desentiende. Puede continuar procesando código de otras aplicaciones. En el momento exacto en que el dispositivo completa la tarea, emite un pulso eléctrico que interrumpe a la CPU forzándola a guardar su estado actual y ejecutar la Rutina de Servicio (ISR) que recuperará el dato de I/O, para luego reanudar donde se había quedado.

7.2. Acceso Directo a Memoria (DMA)

Para cargas masivas de datos (como un disco SSD o puerto Ethernet), el procesador configura un chip secundario especializado llamado Controlador DMA. Este controlador asume el monopolio del bus y transfiere toneladas de bytes desde el dispositivo periférico hasta la memoria RAM sin intermediar en la CPU. Únicamente molestará al procesador principal con una interrupción al finalizar el 100 % del bloque.

8. Análisis y Discusión de los Resultados

8.1. Implicaciones del Modelo Arquitectónico

El enfoque Memory-Mapped I/O unifica el mapeo general fusionando dispositivos y memoria principal. Esta estrategia limpia profundamente el árbol de instrucciones del procesador (acorde a las filosofías RISC y MIPS32) y acelera su ciclo de decodificación. Como compensación de este beneficio, traspasa al programador y al sistema operativo la enorme responsabilidad de demarcar minuciosamente la gestión de la caché y de establecer barreras de acceso estricto, para no provocar desfases temporales en la lectura del hardware.

8.2. Estabilidad y Protección Estructural

El solapamiento de direcciones representa el talón de Aquiles de un sistema MMIO, capaz de desestabilizar críticamente el bus entero. Su contención eficaz exige una barrera multicapa inquebrantable: decodificadores pasivos en la placa base para separar el silicio; la etapa de calibración del firmware UEFI; y, la robusta fiscalización de la Unidad de Gestión de Memoria (MMU), la cual asume la tarea de revocar el acceso físico a programas no privilegiados mediante el bloqueo de permisos por página.

8.3. Maximización del Rendimiento

Finalmente, se establece que dotar a un sistema informático de acceso uniforme a I/O no garantiza rendimiento por sí solo, si la CPU se estanca en rutinas de espera activa. La obligatoriedad del uso de Interrupciones de Hardware y Controladores DMA representa el estándar definitivo para la escalabilidad, pues permite que el núcleo del procesador actúe como un gestor macro en lugar de un operario, delegando los traslados mecánicos del bus y logrando el verdadero paralelismo de Entrada y Salida que exigen los entornos modernos.